SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 Total Marks:30

Annexure A sDry

Run Evaluation

Code of Conduct:

I **SANJAI.S(192511018)** (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

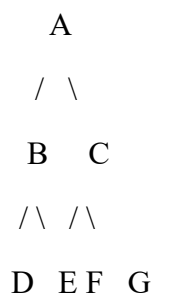
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:harsha

Dry Run Evaluation – Artificial Intelligence

Question 1: Breadth-First Search (BFS)

Given Graph



Algorithm (Breadth-First Search)

1. Start from the root node.
2. Visit the current node.
3. Insert all unvisited neighboring nodes into the queue.
4. Remove the front node from the queue.
5. Repeat until the queue becomes empty.

BFS Dry Run Table

Iteration	Queue Before Expansion	Visited Node	Queue After Expansion
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Answer 1

BFS Traversal Order

A → B → C → D → E → F → G

Queue Contents After Each Iteration

Iteration	Queue
Start	A
After A	B, C
After B	C, D, E
After C	D, E, F, G
After D	E, F, G
After E	F, G
After F	G
After G	Empty

Question 2: Depth-First Search (DFS)

Algorithm (Depth-First Search)

1. Start from the root node.
2. Visit the node.
3. Move to the leftmost unvisited child.
4. Continue until no child exists.
5. Backtrack and visit remaining nodes.

DFS Dry Run Table

Iteration	Stack Before Expansion	Visited Node	Stack After Expansion
1	A	A	C, B
2	C, B	B	C, E, D
3	C, E, D	D	C, E
4	C, E	E	C

Iteration	Stack Before Expansion	Visited Node	Stack After Expansion
5	C	C	G, F
6	G, F	F	G
7	G	G	Empty

DFS Traversal Order

A → B → D → E → C → F → G

Question 3: 8-Puzzle Problem

Initial State **Goal State**

1 2 3

4 5 6

7 8 01 3 6

5 0 2

4 7 8

Breadth-First Search Algorithm 1. Place

the initial state into the queue.

2. Remove the first state.
3. Generate all possible child states.
4. Add unexplored states to the queue.
5. Repeat until the goal state is found.

BFS Dry Run Table

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	Initial State	Initial	Generate 4 child states
2	Child State 1	Remaining 3 + New States	More successors added
3	Child State 2	Queue Updated	Queue Updated
4	Child State 3	Queue Updated	Queue Updated

Iteration	Current State	Queue Before Expansion	Queue After Expansion

5	Continue until Goal Queue Updated	Goal State Found

Answers

1. Which node (state) is expanded in each iteration?

Iteration 1 → Initial State

Iteration 2 → First generated child

Iteration 3 → Second generated child

Iteration 4 → Third generated child

Iteration 5 → Continue level-wise until Goal State

2. How many states are generated in total?

The exact number **depends on the order in which moves (Up, Down, Left, Right) are generated and duplicate-state checking**. Therefore, there is **no single correct numeric answer** from the information given alone.

3. Complete Solution Path

The complete solution path **cannot be uniquely determined** without specifying:

- the move order (Up/Down/Left/Right),
- duplicate checking policy,
- and continuing the BFS until the goal is reached.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Breadth-First Search explores all states level by level. Since every move has the same cost (1), the first time the goal state is reached, it is guaranteed to be the shortest possible sequence of moves. Therefore, BFS always returns the optimal solution in an unweighted 8puzzle.